



TECHNISCHE HOCHSCHULE NÜRNBERG
GEORG SIMON OHM

Datenbanksysteme

Lecture 08

Prof. Dr. Thomas Schedel



Nutzungshinweis

Diese Unterlagen sind **ausschließlich** zu Zwecken der Lehre bestimmt.

Jegliche Vervielfältigung und Verbreitung zu gewerblichen oder anderen Zwecken oder sonst gegen Entgelt sowie die elektronische Zugänglichmachung außerhalb des Intranets der Technischen Hochschule Nürnberg sind **unzulässig**.

Inhalt

- Praxis
 - P7. Subqueries
 - P7.1 Single-row-single-column (Skalare Subqueries)
 - P7.2 Multiple-row-single-column
 - P7.3 Multiple-row-multiple-column
 - P7.4 Korrelierte Subqueries
 - P7.5 EXISTS-Operator
 - P7.6 Anfragen auf Zwischenergebnissen
 - P7.7 Modularisierung (WITH)

<Praxis>

Verschachtelte
Anfrage $\nrightarrow$ nicht erlaubt

(2)

Mitarbeiter	
Name	Gehalt
'Müller'	4000
'Meier'	3000

Name u. Gehalt
aller Mitarbeiter,
die mehr verdienen
als Herr Meier ?

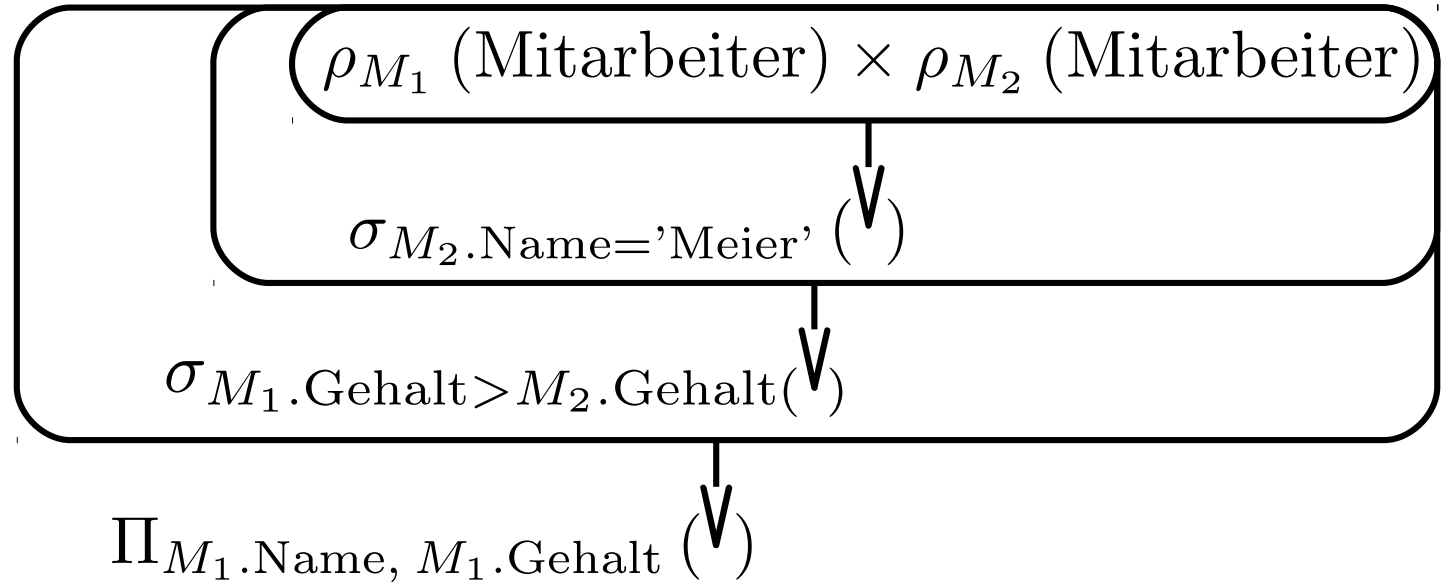
$\Pi_{\text{Name, Gehalt}} (\sigma_{\text{Gehalt} > \Pi_{\text{Gehalt}} (\sigma_{\text{Name} = \text{'Meier'}, (\text{Mitarbeiter}))} (\text{Mitarbeiter})) (\text{Mitarbeiter}))$

WEIL: Vergleich
Skalar > Menge

$\Pi_{M_1.\text{Name}, M_1.\text{Gehalt}} ($
 $\rho_{M_1} (\text{Mitarbeiter})$
 stattdessen
 $\rho_{M_2} (\text{Mitarbeiter}))$
 $\wedge (M_2.\text{Name} = \text{'Meier'}) \wedge (M_1.\text{Gehalt} > M_2.\text{Gehalt})$

1 grosse
Tupelmengen

stufenweise
einschränken



$M_1.\text{Name}$	$M_1.\text{Gehalt}$	$M_2.\text{Name}$	$M_2.\text{Gehalt}$
'Müller'	4000	'Meier'	3000
'Müller'	4000	'Müller'	4000
'Meier'	3000	'Meier'	3000
'Meier'	3000	'Müller'	4000

$\Pi_{M_1.\text{Name}, M_1.\text{Gehalt}} ($

$\rho_{M_1}(\text{Mitarbeiter}) \bowtie_{\substack{M_2.\text{Name} = \text{'Meier'} \\ \wedge \\ M_1.\text{Gehalt} > M_2.\text{Gehalt}}} \rho_{M_2}(\text{Mitarbeiter})$

in SQL
↓

SELECT m1.Name, m1.Gehalt

FROM Mitarbeiter m1 JOIN Mitarbeiter m2

ON (m2.Name = 'Meier' AND m1.Gehalt > m2.Gehalt;

FROM Tabelle name

bsp. FROM Mitarbeiter m1

m1 ist temp. Name für die Tabelle Mitarbeiter

→ Tupelvariable

```
SELECT m1.Name, m1.Gehalt  
FROM Mitarbeiter m1 JOIN Mitarbeiter m2  
ON (m2.Name = 'Meier' AND m1.Gehalt > m2.Gehalt;
```

Join (& Variationen davon) immer nur in der Syntax: !
0

FROM tabelle.spalte JOIN tabelle.spalte

P7. Subqueries

```
SELECT m1.Name, m1.Gehalt  
FROM Mitarbeiter m1 JOIN Mitarbeiter m2  
ON (m2.Name = 'Meier' AND m1.Gehalt > m2.Gehalt;
```

gleicher Effekt
↓

```
SELECT Name, Gehalt  
FROM Mitarbeiter  
WHERE Gehalt > (
```

```
SELECT Gehalt  
FROM Mitarbeiter  
WHERE Name = 'Meier' );
```

Subquery

Immer...

... in runden Klammern

... als rechter Operand

P7.1 Single-row-single-column (Skalare Subqueries)

Bsp. a)



temp
Skalar

bzw. $\{(Skalar)\}$

```
SELECT MitarbeiterNr, Nachname,  
      ( CASE AbteilungsNr
```

```
      WHEN ( SELECT AbteilungsNr  
             FROM Abteilungen  
             WHERE Standort = 'Nürnberg' )
```

(Wenn AbteilungsNr
stimmt, dann 'aus Franken'
sonst "von wo anders")

```
      THEN 'Franken'
```

```
      ELSE 'irgendwo anders' END ) AS "Ort"
```

```
FROM Mitarbeiter;
```

Bsp. b)

```
SELECT MitarbeiterNr, Nachname, Gehalt
FROM Mitarbeiter
WHERE Gehalt > ( SELECT AVG(Gehalt)
                  FROM Mitarbeiter );
```

Bsp. c)

```
SELECT AbteilungsNr, MIN(Gehalt)
FROM Mitarbeiter
GROUP BY AbteilungsNr
HAVING MIN(Gehalt) > ( SELECT MAX(Gehalt)
                        FROM Mitarbeiter
                        WHERE AbteilungsNr = '0815' );
```

Gibt Abteilungen aus, die ein
größeres Mindestgehalt hat
als Abteilung '0815'



```
SELECT MitarbeiterNr, Nachname  
FROM Mitarbeiter  
WHERE Gehalt = ( SELECT MIN(Gehalt)  
                 FROM Mitarbeiter  
                 GROUP BY AbteilungsNr );
```

↓ ↓
Skalar ≠ Menge von Datensätzen
 ↑
undefiniert

P7.2 Multiple-row-single-column

x spezielle Vergleichsoperatoren: [NOT] IN, ANY, ALL

x expr **IN** subquery = true

$\rightarrow \exists \text{obj} \in \text{subquery}: \text{expr} = \text{obj}$

/* kommt Wert überhaupt (nicht) in Ergebnismenge vor? */

mind. einmal enthalten?

x expr operator **ANY** subquery = true

$\rightarrow \exists \text{obj} \in \text{subquery}: \text{expr operator obj} = \text{true}$
operator: =, <>, >, >=, <, <=

/* ^{→ mehr Vergleichsoperatoren als 'IN'} positiver Vergleich mit irgendeinem Element der Ergebnismenge? */

x expr operator **ALL** subquery = true

$\rightarrow \forall \text{obj} \in \text{subquery}: \text{expr operator obj} = \text{true}$

/* positiver Vergleich mit allen Elementen der Ergebnismenge? */

Tipp: (expr IN subquery $\Leftrightarrow$ expr = ANY subquery *)

Bsp.:

```
SELECT MitarbeiterNr, Nachname, Gehalt  
FROM Mitarbeiter
```

```
WHERE Gehalt < ANY  
ALL ( SELECT Gehalt  
FROM Mitarbeiter
```

```
WHERE Job = 'Programmierer' )
```

```
AND Job <> 'Programmierer';
```

→ Arbeiter 'Programmierer' ausnehmen

alle die weniger als
Programmierer verdienen

ANY : Mitarbeitergehalt < mind. einem Programmierergehalt

ALL : Mitarbeitergehalt < aller Programmierergehälter

→ Ergebnis: Mitarbeiter, die weniger verdienen,
als (irgend) ein Programmierer !

Aufgabenstellung:

Finde alle Mitarbeiter,
die mit einem Hans in derselben Abteilung sitzen
und
auch denselben Vorgesetzten haben wie dieser Hans



P7.3 Multiple-row-multiple-column

SELECT MitarbeiterNr

FROM Mitarbeiter

WHERE Abteilung IN (SELECT Abteilung
FROM Mitarbeiter
WHERE Vorname = 'Hans')

AND Vorgesetzter IN (SELECT Vorgesetzter
FROM Mitarbeiter
WHERE Vorname = 'Hans')

AND Vorname <> 'Hans';

... (Abteilung, Vorgesetzter) IN

(SELECT Abteilung, Vorgesetzter
FROM Mitarbeiter
WHERE Vorname = 'Hans')



1a



Vorname 'Hans'	
Vorgesetzte	Abteilung
1	A
2	B
3	C

②



 $\left\{ \begin{matrix} 1 \\ 2 \\ 3 \end{matrix} \right\}$



 $\left\{ \begin{matrix} A \\ B \\ C \end{matrix} \right\}$

$\times$

→ 9 Kombinationen
 → nur 3 zulässig

Aufgabenstellung:

Finde alle Mitarbeiter,
die in ihrer Abteilung
überdurchschnittlich viel verdienen



P7.4 Korrelierte Subqueries

im Grunde wie "this.variable" beim Programmieren

Bsp.:

```
SELECT MitarbeiterNr  
FROM Mitarbeiter outer_table  
WHERE Gehalt > ( SELECT AVG(Gehalt)  
                  FROM Mitarbeiter inner_table  
                  WHERE inner_table.Abteilung = outer_table.Abteilung );
```

⇒ Für jede einzelne Zeile der äusseren Abfrage
muss die innere Abfrage neu ausgewertet werden !

P7.5 EXISTS-Operator

... WHERE [NOT] EXISTS subquery



liefert subquery (keine) Ergebnisse ?

P7.6 Anfragen auf Zwischenergebnissen

SELECT s.A, s.E

FROM ((SELECT A,B,C FROM T1 WHERE p)

NATURAL JOIN

(SELECT D AS C, E, F FROM T2 WHERE q)) s;

temp Name 's'

↓ entspricht

$$\Pi_{s.A, s.E} \left(\rho_s \left(\left(\Pi_{A,B,C} \left(\sigma_p(T_1) \right) \right) \bowtie \left(\rho_{C \leftarrow D} \left(\Pi_{D,E,F} \left(\sigma_q(T_2) \right) \right) \right) \right) \right)$$

P7.7 Modularisierung (WITH)

```
WITH query1 AS ( SELECT . . . ),  
     query2 AS ( SELECT . . . FROM query1 . . . ),  
     .  
     .  
     .  
     queryn AS ( SELECT . . . )  
SELECT . . .  
FROM query1  
WHERE . . . < query2  
GROUP BY . . .  
HAVING . . . > queryn
```

Vorteile:

- + kein redundanter Abfrage-Code
- + Speicherung der Zwischenergebnisse

</Praxis>



Datenbanksysteme

Lecture 08

`exit(0);`

